



**Кафедра информационных систем  
в химической технологии**

**Институт тонких химических технологий  
им. М.В. Ломоносова  
РТУ МИРЭА**



**ПРОГРАММИРОВАНИЕ В ЗАДАЧАХ ХТ**

# **Лекция 4. Управляющие конструкции**

# Содержание лекции

---

В данной лекции будут рассмотрены следующие темы:

-  Условные операторы
-  Операторы сравнения
-  Тернарный оператор
-  Циклы
-  Сортировка данных
-  Списковые включения
-  Генераторы

# Условные операторы (1 из 3)

📡 Смысл конструкции `if-else` – “если – то”

📡 Данная конструкция состоит из:

- ⦿ Оператор `if`
- ⦿ Условия
- ⦿ Оператор `elif`
- ⦿ Оператор `else`
- ⦿ Исполняемые выражения

Операторы `elif` и `else`  
опциональны

Условия

```
if a > 0:
```

```
    print("A > 0")
```

```
elif a < 0:
```

```
    print("A < 0")
```

```
else:
```

```
    print("A = 0")
```

Исполняемые  
выражения

# Условные операторы (2 из 3)

- Выражение выполняется только в том случае, если оно указано после оператора **else** или, если предшествующее условие истинно
- Может быть исполнено только одно выражение
  - Далее происходит выход из условного блока

Условие истинно

```
a = 5
if a > 0:
    print("A > 0")
elif a > 1:
    print("A > 1")
```

**Вывод:** "A > 0"

Выражение  
исполнено

- Числа, не равные нулю и непустые объекты – **True**
- 0 и пустые объекты – **False**

# Условные операторы (3 из 3)

- 📡 Условия могут быть сложными
- 📡 Для составления сложных условий используются операторы:
  - ⦿ `a and b` – возвращает True, если `a` и `b` истинны
  - ⦿ `a or b` – возвращает True, если `a` или `b` истинно
  - ⦿ `not a` – возвращает True, если `a` ложно; и наоборот

```
a = 5  
b = -1
```

```
if a > 3 and not (b > 0):  
    print("True!")
```

**Вывод:** "True!"

Сложное условие. Будет True, если `a` больше 3 и не `b` больше 0 (`b` меньше либо равно 0)

# Операторы сравнения

☞ Существует 2 способа сравнения:

- ☉ Через оператор "=="
- ☉ Через оператор "is"

☞ Оператор "==" сравнивает значения, а оператор "is" проверяет ссылаются ли сравниваемые переменные на один и тот же объект

```
a = "Hello"
b = "Hello"
print(id(a))    140088216522400
print(id(b))    140088216522400
print(a == b)   True
print(a is b)   True
```

```
a = "Hello"
b = "Hello" + "l"
b = b[:len(b)-1]
print(b)        "Hello"
print(id(a))    140190225960608
print(id(b))    140190225301832
print(a == b)   True
print(a is b)   False
```

# Тернарный оператор

- Тернарный оператор – более короткий аналог условного выражения
- Структура: (выражение) **if** (условие) **else** (выражение)

```
if a > 5:  
    a = 10  
else:  
    a = 0
```

```
a = 6  
a = 10 if a > 5 else a = 0  
print(a) Вывод: 10
```

- Сокращенный тернарный оператор

```
True or print("Hello world")  
False or print("Hello world")
```

Подходит для быстрых проверок значения переменной

# Оператор сопоставления с шаблоном

- Оператор `match` сопоставляет результат выражения, заданного после него с шаблонами, определенными после `case`
  - Если выражение подходит под шаблон, выполняется блок кода, следующий за соответствующим `case`
  - Дополнительно может проверяться условие, определенное оператором `if`, следующим за шаблоном

```
flag = False
match (100, 200):
    case (100, 300): # Несовпадение: 200 != 300
        print('Case 1')
    case (100, 200) if flag: # Совпадение, но условие не выполнено
        print('Case 2')
    case (100, y): # Совпадение и присвоение у значения 200
        print(f'Case 3, y: {y}')
    case _: # Сопоставления не проводилось
        print('Case 4, I match anything!')
# Case 3, y: 200
```

# Циклы (1 из 8)

- ☞ Существует 2 способа организовать цикл:
  - ☉ Цикл `for ... in ...`
  - ☉ Цикл `while`
- ☞ Цикл `for ... in ...` выполняется заданное число раз
- ☞ Цикл `while` выполняется до тех пор, пока выполняется условие
- ☞ Оператор `else` определяет код, который должен быть выполнен, если цикл завершился обычным образом

```
for i in range(0, 5):  
    print(i)
```

```
i = 0  
while i < 5:  
    print(i)  
    i += 1
```

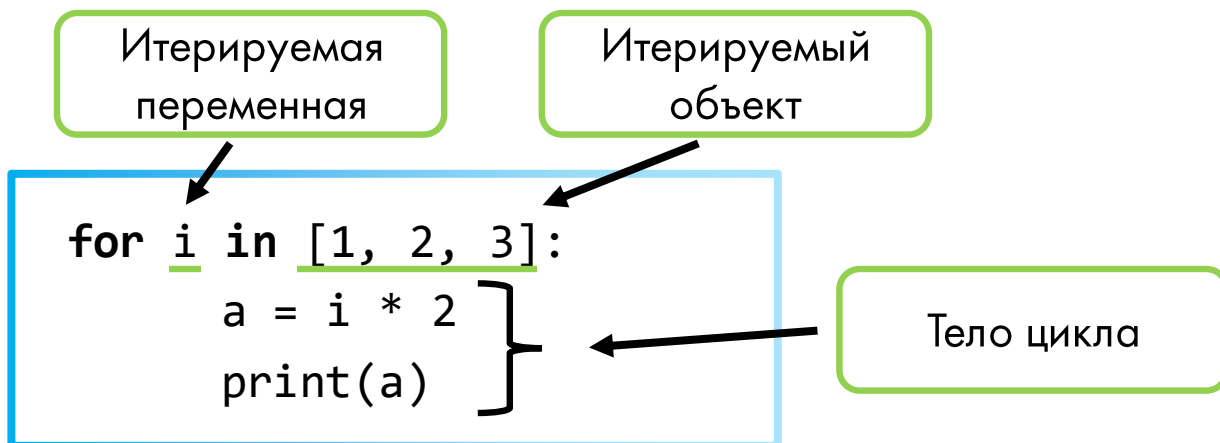
# Циклы (2 из 8)

📶 Цикл `for ... in ...` выполняется заданное число раз

📶 Компоненты цикла:

- 🕒 Операторы `for`, `in`
- 🕒 Итерируемая переменная
- 🕒 Итерируемый объект
- 🕒 Тело цикла

📶 Итерируемая переменная последовательно принимает значения из итерируемого объекта



# Циклы (3 из 8)

---

 Первая итерация:

- ⦿ `i = 1`
- ⦿ `a = 2`
- ⦿ `print(a)` – выводит 2

 Вторая итерация:

- ⦿ `i = 2`
- ⦿ `a = 4`
- ⦿ `print(a)` – выводит 4

 Третья итерация:

- ⦿ `i = 3`
- ⦿ `a = 6`
- ⦿ `print(a)` – выводит 6

```
for i in [1, 2, 3]:  
    a = i * 2  
    print(a)
```

# Циклы (4 из 8)

📶 Цикл **while** выполняется до тех пор, пока выполняется условие

📶 Компоненты цикла:

- ⦿ Оператор while
- ⦿ Условие продолжения цикла
- ⦿ Итерируемая переменная
- ⦿ Тело цикла
- ⦿ Изменение значения итерируемой переменной

Итерируемая  
переменная

Условие

Тело цикла

```
i = 1
```

```
while
```

```
i < 4:
```

```
    a = i * 2
```

```
    print(a)
```

```
    i += 1
```

Если не изменять значение  
итерируемой переменной, то  
начнется бесконечный цикл

# Циклы (5 из 8)

📡 Операторы **break** и **continue** предназначены для управления циклами

- ⦿ **break** досрочно завершает цикл
- ⦿ **continue** осуществляет переход к следующей итерации

Переход к следующей итерации при  $i = 2$

```
for i in [1, 2, 3]:  
    if i == 2:  
        continue  
    print(i)
```

**Вывод:** 1, 3

Выход из цикла при  $i = 2$

```
for i in [1, 2, 3]:  
    if i == 2:  
        break  
    print(i)
```

**Вывод:** 1

# Циклы (6 из 8)

- Оператор **else** определяет блок кода, который будет выполнен, если цикл обычным образом (без использования **break**)
- Используется в циклах **for ... in ...** и **while**

```
for i in range(1, 4):  
    if i == 20:  
        print("Цикл завершился досрочно!")  
        break  
else:  
    print("Цикл завершился обычным образом!")
```

**Вывод:** “Цикл завершился обычным образом!”

# Циклы (7 из 8)

- В циклах часто используются функции `len()` и `range()` или их комбинация
  - Функция `len()` возвращает количество элементов в итерируемом объекте
  - Функция `range()` позволяет создать последовательность из чисел
- Функция `range()` вызывается в трёх вариантах:
  - `range(a)` – числа от 0 до `a - 1`
  - `range(a, b)` – числа от `a` до `b - 1`
  - `range(a, b, step)` – числа от `a` до `b - 1` с шагом `step`

```
list_1 = [1, 2, 3]
for i in range(len(list_1)):
    print(i)
```

**Вывод:** 0, 1, 2

```
for i in range(-1, -4, -1):
    print(i)
```

**Вывод:** -1, -2, -3

# Циклы (8 из 8)

- Функция `enumerate()` позволяет перебирать итерируемые объекты, получая для каждого элемента кортеж, состоящий из индекса элемента и самого элемента
- Используется множественное присвоение

`i = (index, element)`

```
list_1 = [1, 2, 3]
for i in enumerate(list_1):
    print(i)
```

**Вывод:**

```
(0, 1)
(1, 2)
(2, 3)
```

`i, e = (index, element)`

```
list_1 = [1, 2, 3]
for i, e in enumerate(list_1):
    print(i, e)
```

**Вывод:**

```
0 1
1 2
2 3
```

# Сортировка данных (1 из 5)

- 📡 Сортировки предназначены для упорядочивания элементов итерируемого объекта по какому-либо принципу
- 📡 Встроенная функция `sorted()` предназначена для сортировки любых итерируемых объектов
  - 🕒 Данный метод возвращает новый объект
- 📡 У списков есть метод `.sort()`
  - 🕒 Данный метод изменяет исходный объект

```
list_1 = [3, 1, 2]
print(sorted(list_1))
```

```
list_1.sort()
print(list_1)
```

**Вывод:** [1, 2, 3]      [1, 2, 3]

# Сортировка данных (2 из 5)

- Изменить стандартный принцип сортировки можно при помощи параметра **key** функций **sorted** и **.sort()**
- В данном параметре указывается функция, которая вызывается для каждого элемента сортируемого объекта и возвращает значение, по которому нужно проводить сортировку
  - Функция должна принимать один аргумент

```
def sort_by_second(number):  
    second = str(number)[1]  
    return second  
  
list_1 = [37, 42, 55]  
print(sorted(list_1, key = sort_by_second))
```

**Вывод:** [42, 55, 37]

Функция не  
вызывается, а просто  
указывается

# Сортировка данных (3 из 5)

- 🌀 Без использования параметра `key` сложно будет обойтись при сортировке сложных объектов
- 🌀 При сортировке удобно использовать `lambda`-функции

```
results = [  
    ["Матем.", 65, 98],  
    ["Русс. яз.", 71, 99],  
    ["Общ.", 67, 95]  
]
```

```
print(sorted(results))
```

**Вывод:**

```
[['Матем.', 65, 98], ['Общ.', 67, 95], ['Русс. яз.', 71, 99]]
```

```
print(sorted(results, key = lambda result: result[2]))
```

**Вывод:**

```
[['Общ.', 67, 95], ['Матем.', 65, 98], ['Русс. яз.', 71, 99]]
```

# Сортировка данных (4 из 5)

- Модуль `operator` предлагает функции `itemgetter()`, `attrgetter()` и `methodcaller()` для организации типовых сортировок
  - Функции `attrgetter()` и `methodcaller()` применяются при сортировке объектов пользовательских классов
  - Функция `itemgetter()` позволяет удобнее выбирать значения, по которым происходит сортировка
  - Также данная функция позволяет организовать многоуровневую сортировку

```
from operator import itemgetter
results = [{"Математика", 65, 98}, ...]
print(sorted(results, key = itemgetter(2)))
```

**Вывод:**

```
[['Общ.', 67, 95], ['Матем.', 65, 98], ['Русс. яз.', 71, 99]]
```

# Сортировка данных (5 из 5)

- 🌀 Устройство функций сортировки позволяет проводить сложные составные сортировки
- 🌀 Сначала проводится одна сортировка с одним ключом, потом проводится вторая сортировка со вторым ключом

```
from operator import itemgetter
results = [
    ["Ваня", 65],
    ["Петя", 71],
    ["Ваня", 67]
]
results = sorted(results, key = itemgetter(0))
print(results) Вывод: [['Ваня', 65], ['Ваня', 67], ['Петя', 71]]

results = sorted(results, key = itemgetter(1))
print(results) Вывод: [['Ваня', 67], ['Ваня', 65], ['Петя', 71]]
```

# Списковые включения (1 из 3)

- 📡 Списковые включения позволяют создавать списки на основе итерируемых объектов
- 📡 Списковые включения могут содержать вложенные циклы и условия
- 📡 Структура: [`<expr>` for `<elem>` in `<iterable>` if `<condition>` ]

В список `list_2` добавить квадраты элементов списка `list_1`, которые больше 1



```
list_1 = [1, 2, 3]
list_2 = [number ** 2 for number in list_1 if number > 1]
print(list_2)
Вывод: [4, 9]
```

# Списковые включения (2 из 3)

 Каждое списковое включение можно заменить эквивалентом

```
list_1 = [['H', 'e'], ['l', 'l'], ['o']]
list_2 = [letter for sub_list in list_1 for letter in sub_list]
print("".join(list_2))
```

**Вывод:** "Hello"

```
list_1 = [['H', 'e'], ['l', 'l'], ['o']]
list_2 = []
for sub_list in list_1:
    for letter in sub_list:
        list_2.append(letter)
print("".join(list_2))
```

**Вывод:** "Hello"

# Списковые включения (3 из 3)

- 📡 Списковые включения могут быть использованы для создания словарей и множеств
- 📡 Главное отличие – фигурные скобки
- 📡 Для создания словаря используется соответствующий «словарный» синтаксис

```
a = {number: chr(number) for number in range(65, 91)}  
print(a)
```

**Вывод:** {65: 'A', 66: 'B', ..., 89: 'Y', 90: 'Z'}

```
a = {chr(number) for number in range(65, 91)}  
print(a)
```

**Вывод:** {'X', 'H', 'I', ..., 'T', 'Q', 'N'}

# Генераторы (1 из 4)

- Генератор – функция, вызов которой не исполняет её, а создает итерируемый объект-генератор
- Генераторы эффективнее списков, так как используют ленивые вычисления
  - Вычисление выражения откладывается до тех пор, пока его значение не будет востребовано
- Вместо оператора `return` используется оператор `yield`
  - Оператор `yield` приостанавливает функцию
  - Функция `next()` выполняет функцию до следующего оператора `yield`

```
def gen():  
    yield 1  
    yield 2  
  
generator = gen()
```

```
print(next(generator)) 1  
print(next(generator)) 2
```

# Генераторы (2 из 4)

📡 Генератор можно перебирать при помощи:

- ⦿ Функции `next()`
- ⦿ Циклов

📡 Использование циклов аналогично цепочке вызовов функции `next()`

```
def gen():  
    yield 1  
    yield 2  
    yield 3  
generator = gen()  
  
print(next(generator)) 1  
print(next(generator)) 2  
print(next(generator)) 3
```

```
def gen():  
    yield 1  
    yield 2  
    yield 3  
generator = gen()  
  
for i in generator:  
    print(i)  
Вывод: 1, 2, 3
```

# Генераторы (3 из 4)

📡 Генераторы можно создавать в стиле списковых включений

- ⦿ Вместо квадратных используются круглые скобки
- ⦿ Такая конструкция будет создавать генератор, а не список

📡 Генератор можно перебирать только **один раз**

```
list_1 = [i for i in range(1, 4)]
```

```
for i in list_1:  
    print(i)
```

**Вывод:** 1, 2, 3

Это список

```
generator = (i for i in range(1, 4))
```

```
for i in generator:  
    print(i)
```

**Вывод:** 1, 2, 3

Это генератор

# Генераторы (4 из 4)

- Списки после создания целиком помещаются в оперативную память

Оперативная  
память



- При использовании генератора в памяти хранится только одно значение

Оперативная  
память



- Очередное значение попадает в ОП после удаления оттуда предыдущего значения

Оперативная  
память



# Итоги

---

В данной лекции были рассмотрены следующие темы:

-  Условные операторы
-  Операторы сравнения
-  Тернарный оператор
-  Циклы
-  Сортировка данных
-  Списковые включения
-  Генераторы